

Informe del control 2 de Redes (CC4303)

Curso: Redes (CC4303), primavera 2026

Profesora: Ivana Bachmann

Integrante: Diego Salinas

Repositorio: [redes-diego-salinas-2026](#)

Fecha: 4 de octubre de 2026

Alcance del control

El control 2 reúne dos actividades que construyen un Transmission Control Protocol (TCP) simplificado sobre User Datagram Protocol (UDP), y ambas transfieren archivos íntegros bajo pérdida inducida. La actividad 3 implementa la clase SocketTCP con Stop & Wait, mientras que la actividad 4 agrega Go-Back-N y control de congestión al estilo de TCP Tahoe. Las mediciones muestran que el control de congestión recorta cerca del 40% de los segmentos intercambiados, aunque alarga el tiempo total de la transferencia.

El documento recorre el flujo de ambos códigos, la forma de ejecutarlos y las decisiones de diseño que el enunciado dejó abiertas. También incluye los diagramas pedidos en los pasos 4 y 7 de la actividad 3, las respuestas de la parte 2 de la actividad 4 y los experimentos de la sección de pruebas. Las pruebas se ejecutaron sobre Windows Subsystem for Linux (WSL) con Ubuntu y Python 3.12, con pérdidas simuladas en el propio código y también con netem.

Ejecución de los códigos

Los códigos de la actividad 3 viven en la carpeta `control_2/actividad_3` y los de la actividad 4 en `control_2/actividad_4`, cada carpeta con su clase SocketTCP, su cliente y su servidor. La actividad 4 suma la clase CongestionControl y utiliza sin cambios los módulos `socketUDP` y `slidingWindowCC` entregados por el curso. Ninguna de las dos carpetas necesita librerías externas, basta con Python 3 y los módulos estándar `socket`, `random`, `sys` y `time`.

El servidor se levanta primero y el cliente recibe por entrada estándar el archivo que va a transferir. El servidor escribe en salida estándar solo el contenido recibido y manda los mensajes informativos a error estándar, de modo que la salida se puede redirigir y comparar con el original. Las opciones de línea de comandos eligen el protocolo, activan el control de congestión, fijan el *timeout* y ajustan la tasa de pérdida simulada.

```
cd control_2/actividad_4
python3 servidor.py localhost 8000 -mode go_back_n -timeout 0.5 > recibido.txt
python3 cliente.py localhost 8000 -mode go_back_n -cc on -perdida 20 < archivo.txt
cmp archivo.txt recibido.txt
```

El modo de depuración se activa con la opción `-debug` e imprime en error estándar los *timeout*, las retransmisiones, los segmentos repetidos y la evolución de la ventana de congestión. Los tests del enunciado se ejecutan con `pruebas_servidor.py` y `pruebas_cliente.py`, que aceptan las mismas opciones y recorren los cinco casos de envío y recepción. El experimento de comparación se reproduce con `experimento.py`, que repite cada configuración cinco veces y guarda las mediciones en un archivo separado por comas.

Formato de los segmentos

Los headers se codifican directamente en bytes y ocupan un tamaño fijo de 5 B, con lo que separar el área de datos resulta inmediato. El primer byte lleva las banderas de sincronización, confirmación y término, conocidas como SYN, ACK y FIN, y los cuatro siguientes guardan el número de secuencia. La Tabla 1 resume esa estructura, que las dos actividades comparten porque el receptor debe interpretar los mismos campos.

Bytes	Campo	Contenido
0	Banderas	bit 0 para SYN, bit 1 para ACK, bit 2 para FIN
1 a 4	Secuencia	entero sin signo de 4 B en orden de red
5 en adelante	Datos	16 B con Stop & Wait y 8 B con Go-Back-N

Tabla 1: estructura de un segmento

Las funciones estáticas `parse_segment` y `create_segment` traducen entre los bytes del segmento y un diccionario con las llaves de las tres banderas, la secuencia y los datos. El socket UDP interno trabaja con un buffer fijo de 21 B, que cubre el header más los 16 B de datos del caso más grande. El número de secuencia avanza según los bytes transmitidos, de modo que una confirmación con secuencia igual a la del segmento más su largo confirma ese segmento.

Establecimiento de la conexión

El *3-way handshake* sigue la versión simplificada del curso, con un único número de secuencia que ambos extremos mantienen sincronizado. El cliente elige un valor aleatorio entre 0 y 100 para el SYN, el servidor responde SYN+ACK con ese valor más uno y el cliente cierra con un ACK que suma dos. La Figura 1 asocia cada paso con las funciones `connect` y `accept`, incluida la creación del socket que atiende la conexión.

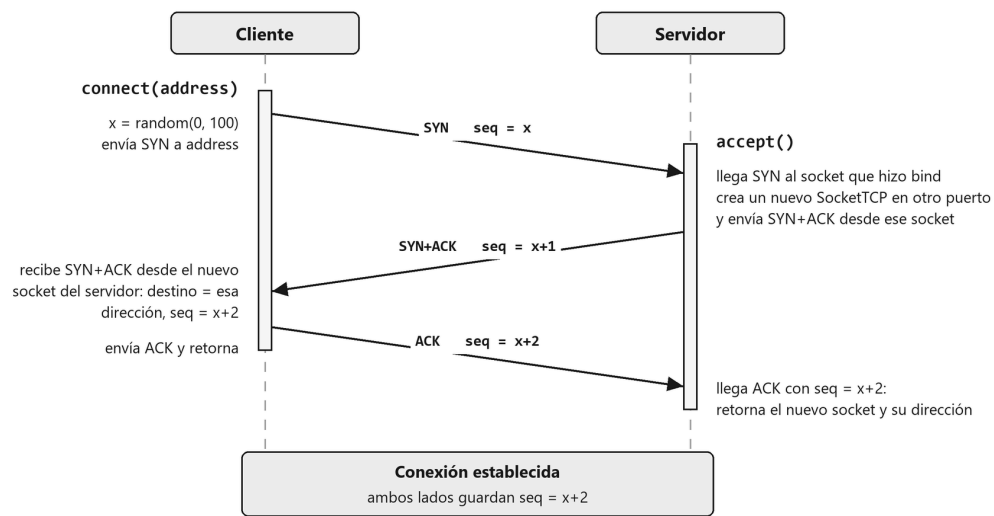


Figura 1: pasos del 3-way handshake en `connect` y `accept`

El socket que retorna `accept` se asocia con `bind` al puerto 0, de modo que el sistema operativo le entrega un puerto libre distinto al del socket que escucha. El SYN+ACK sale desde ese socket nuevo, así que el cliente aprende la dirección definitiva leyendo el origen del datagrama que recibe. Ante pérdidas,

connect reenvía el SYN y accept reenvía el SYN+ACK cada vez que vence el *timeout*, sin más coordinación que esos reenvíos.

Pérdida del último ACK del handshake

El paso 7 de la actividad 3 pide analizar qué ocurre cuando se pierde el ACK que cierra el handshake, caso que deja a los dos extremos en estados distintos (Bachmann, 2026a). El cliente ya considera establecida la conexión y empieza a enviar datos, mientras que accept sigue esperando ese ACK y reenvía SYN+ACK. Sin un tratamiento explícito cada lado ignora lo que manda el otro y la conexión queda detenida, como ilustra la Figura 2.

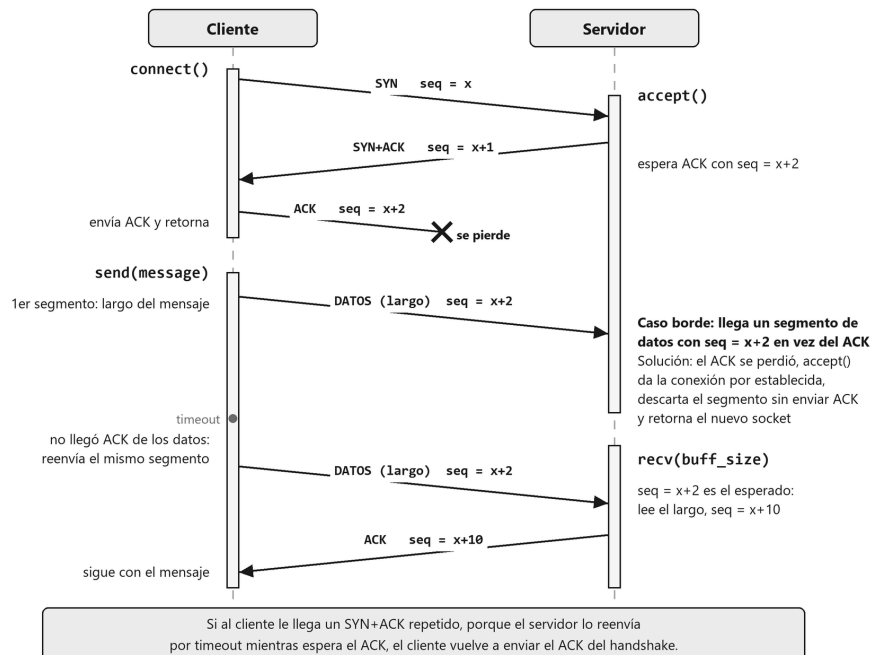


Figura 2: pérdida del último ACK del handshake y su solución

La solución adoptada identifica ese segmento dentro de `accept`, porque un segmento sin banderas con la secuencia que sigue al handshake solo puede venir de un cliente que ya lo terminó. En ese caso `accept` da por establecida la conexión y descarta el segmento sin responder, de modo que el cliente lo reenvía por *timeout* y lo recibe `recv` de forma habitual. Guardar ese segmento entre funciones habría ahorrado una espera, pero obliga a arrastrar estado entre `accept` y `recv`.

Un segundo mecanismo cubre el caso en que el cliente no envía nada después de conectarse y queda a la espera de datos del servidor. Cuando llega un SYN+ACK repetido, el cliente vuelve a enviar el ACK del handshake desde cualquier punto en que esté esperando. Ambos mecanismos juntos cierran el caso borde sin alterar el formato de los segmentos ni la numeración acordada.

Transferencia con Stop & Wait

La función `send` divide el mensaje en trozos de 16 B y antepone un segmento con el largo total, para que el receptor sepa cuándo dejar de esperar. Cada trozo viaja solo y el emisor lo reenvía mientras no llegue la confirmación con el número de secuencia esperado, apoyado en el *timeout* del socket. El receptor acepta el segmento en orden, avanza su secuencia y reenvía el último ACK cuando reconoce una copia repetida.

La función `recv` mantiene dos variables de estado, los bytes del mensaje que faltan por llegar y los datos recibidos que todavía no se entregan. Con ambas en cero la llamada espera primero el segmento de largo, y en caso contrario continúa un mensaje que quedó a medias. El retorno ocurre apenas se junta el mínimo entre `buff_size` y lo que resta del mensaje, y el excedente queda guardado para la llamada siguiente.

Cierre de la conexión

La función `close` envía un FIN y espera el FIN+ACK correspondiente, con un reenvío del FIN en cada *timeout* hasta un máximo de tres. Si recibe la respuesta manda el último ACK tres veces con una espera intermedia, y si agota los tres plazos asume que la contraparte ya se cerró. La función `recv_close` hace el papel inverso, responde FIN+ACK a los FIN repetidos y también se rinde después del tercer vencimiento.

Control de congestión

La clase `CongestionControl` concentra el estado del control de congestión y deja a `SocketTCP` el manejo de los segmentos. Guarda el tamaño máximo de datos por segmento, conocido como Maximum Segment Size (MSS), la ventana de congestión en `cwnd` y el umbral en `ssthresh`. La ventana parte en un MSS, el umbral empieza indefinido y el estado inicial es *slow start*, tal como exige el test entregado por el curso.

Las dos funciones de evento concentran el comportamiento de la clase en pocas líneas. La primera atiende la llegada de una confirmación y la segunda el vencimiento del plazo de espera, y entre ambas resuelven el cambio de estado. El resto de la clase son consultas que `SocketTCP` utiliza para dimensionar la ventana de envío.

```
def event_ack_received(self) -> None:
    if self.is_state_slow_start():
        self.cwnd += self.MSS
        if self.ssthresh is not None and self.cwnd >= self.ssthresh:
            self.current_state = CONGESTION_AVOIDANCE
    else:
        self.cwnd += self.MSS / self.get_MSS_in_cwnd()

def event_timeout(self) -> None:
    self.ssthresh = int(self.cwnd / 2)
    self.cwnd = self.MSS
    self.current_state = SLOW_START
```

En *slow start* cada confirmación suma un MSS completo, de modo que la ventana se duplica una vez por ronda. En *congestion avoidance* el incremento equivale a la fracción de MSS que corresponde a la ventana actual, el aumento aditivo que describen Kurose y Ross (2013). Cada *timeout* fija el umbral en la mitad entera de la ventana y la devuelve a un MSS, lo que reproduce la disminución multiplicativa de TCP Tahoe.

El test `CongestionControl_test.py` entregado por el curso pasa sus cinco casos con el MSS de 7 B que trae por defecto. La clase también se probó con valores de 1, 4, 8 y 13 B, y en esos casos los cinco controles volvieron a resultar correctos. El umbral se calcula con la parte entera de la mitad de la ventana, detalle que el test verifica al comparar contra `int(5/2*MSS)`.

Go-Back-N sobre la ventana deslizante

La función `send_using_go_back_n` arma la lista de datos con el largo del mensaje y los trozos de 8 B, y entrega esa lista a un objeto `SlidingWindowCC`. El tamaño inicial de la ventana sale del objeto

CongestionControl cuando el control está activo, y en caso contrario queda fijo en ocho segmentos. Los elementos de esa ventana salen de inmediato y el primer envío arranca el único timer que ofrece SocketUDP.

Cada confirmación acumulativa mueve la ventana tantas posiciones como elementos reconoce, y los que entran se envían enseguida. Un *timeout* reenvía la ventana completa, la respuesta clásica de Go-Back-N, después de que el control de congestión reduce su tamaño. El emisor ignora las confirmaciones que no mueven la ventana, porque esta versión simplificada no implementa *fast retransmit*.

El enunciado advierte que la ventana puede encogerse justo después de enviar su último elemento, lo que deja llegar una confirmación por delante de todo lo que contiene (Bachmann, 2026b). La función `avanzar_ventana` resuelve ese caso moviendo la ventana completa y repitiendo la revisión hasta que la confirmación cae dentro o deja de avanzar. La misma rutina sirve para el caso normal, de modo que no hay dos caminos distintos que mantener.

```
def avanzar_ventana(self, data_window, window_size, ack_seq):
    pasos_totales = 0
    while True:
        pasos = 0
        for indice in range(window_size):
            seq_elemento = data_window.get_sequence_number(indice)
            if seq_elemento is None or seq_elemento >= ack_seq:
                break
            pasos += 1
        if pasos == 0:
            return pasos_totales
        data_window.move_window(pasos)
        pasos_totales += pasos
        if pasos < window_size:
            return pasos_totales
```

El receptor de Go-Back-N acepta únicamente el segmento que continúa su secuencia y responde con la confirmación acumulativa correspondiente. Cualquier segmento fuera de orden se descarta y provoca el reenvío del último ACK, que es lo que mantiene sincronizado al emisor. Ese comportamiento convive con el receptor de Stop & Wait porque ambos comparten el formato de segmento y la numeración por bytes.

Relación entre Go-Back-N y Stop & Wait

La primera pregunta de la parte 2 apunta a por qué conviene partir del código de Stop & Wait, y la respuesta es que ese protocolo equivale a Go-Back-N con ventana de un segmento. Ambos comparten el formato de los segmentos, la confirmación acumulativa, la retransmisión por *timeout* y el avance de la secuencia según los bytes enviados. La diferencia está en cuántos segmentos quedan sin confirmar a la vez, así que basta con ampliar la ventana para obtener el otro protocolo.

La segunda pregunta indaga qué ocurre con `recv`, que el material solo muestra para el emisor. El receptor cambia poco al adoptar SocketUDP y SlidingWindowCC, porque no necesita ventana ni timers, solo reconocer el segmento en orden y responder la confirmación acumulativa. En esta implementación la única diferencia entre ambos modos de recepción es qué hacer con lo que llega fuera de orden, descartarlo en vez de tratarlo como copia repetida.

Decisiones de diseño

El largo del mensaje viaja como entero de 8 B en el área de datos del primer segmento, lo que mantiene el header fijo y simplifica el análisis. Las pérdidas simuladas se aplican en la recepción y no en el envío,

porque el timer de SocketUDP solo arranca al enviar y un envío omitido dejaría al emisor sin ningún despertador. Esa decisión además se parece más a netem, donde el datagrama sí se transmite y se pierde dentro de la red.

La función `recv` retorna una cadena vacía cuando llega un FIN, igual que los sockets orientados a conexión de Python. El servidor aprovecha ese contrato para recibir archivos de largo desconocido llamando a `recv` hasta que la cadena venga vacía. El *timeout* por defecto es de 1,5 s, algo mayor que el tiempo de ida y vuelta de 1 s que impone netem con medio segundo de retardo por sentido.

Casos borde encontrados

El timer de SocketUDP no se detiene al recibir un mensaje, de modo que cada espera exitosa obliga a detenerlo a mano antes de continuar. Ese detalle provocó un bloqueo cuando el emisor atendía un SYN+ACK repetido en medio de Go-Back-N, porque detenía el timer y se quedaba sin nada que lo despertara. La versión final conserva el timer corriendo en ese camino y solo lo reinicia al retransmitir la ventana.

Cuando se pierde la confirmación del último trozo, el receptor ya retornó de `recv` y el emisor insiste con la copia. La llamada siguiente a `recv` o `recv_close` atiende esa copia y reenvía el ACK, así que el receptor debe seguir leyendo después de completar un mensaje. Si en cambio se pierden los tres FIN de `close`, el emisor se cierra igual y la contraparte queda esperando, comportamiento que el enunciado acepta.

Pruebas realizadas

Los cinco tests del enunciado pasaron en los cuatro escenarios probados, con y sin control de congestión, sin pérdida y con un 20% de pérdida. La prueba del paso 5 confirmó el reparto esperado, ya que un mensaje de 19 B se entrega como 14 B y 5 B cuando `buff_size` vale 14. Un mensaje de 42 B leído con dos llamadas de 21 B entregó 21 B en cada una, con un tamaño que no es múltiplo del segmento.

Las transferencias de archivo llegaron íntegras en las veinte corridas del experimento y en las pruebas manuales con pérdidas del 20% al 40%. La actividad 3 se verificó además con netem configurado en 20% de pérdida y medio segundo de retardo, donde el archivo de 542 B llegó sin diferencias. La actividad 4 repitió esa prueba en modo Go-Back-N y entregó el archivo de 100 B completo en 21,3 s.

Comparación con y sin control de congestión

El experimento final transfirió archivos de 100 B y de 1.200 B con un 20% de pérdida, un *timeout* de 0,5 s y cinco repeticiones por configuración. La Tabla 2 reúne los promedios de tiempo y de segmentos intercambiados entre cliente y servidor, incluidos los reenvíos y las confirmaciones. La Figura 3 compara esos promedios y deja ver que las dos métricas apuntan en direcciones opuestas.

Archivo	Control de congestión	Tiempo medio	Segmentos medios
100 B	activado	2,91 s	50,6
100 B	desactivado	2,74 s	76,0
1.200 B	activado	21,87 s	469,6
1.200 B	desactivado	18,09 s	774,8

Tabla 2: promedios de cinco repeticiones por configuración

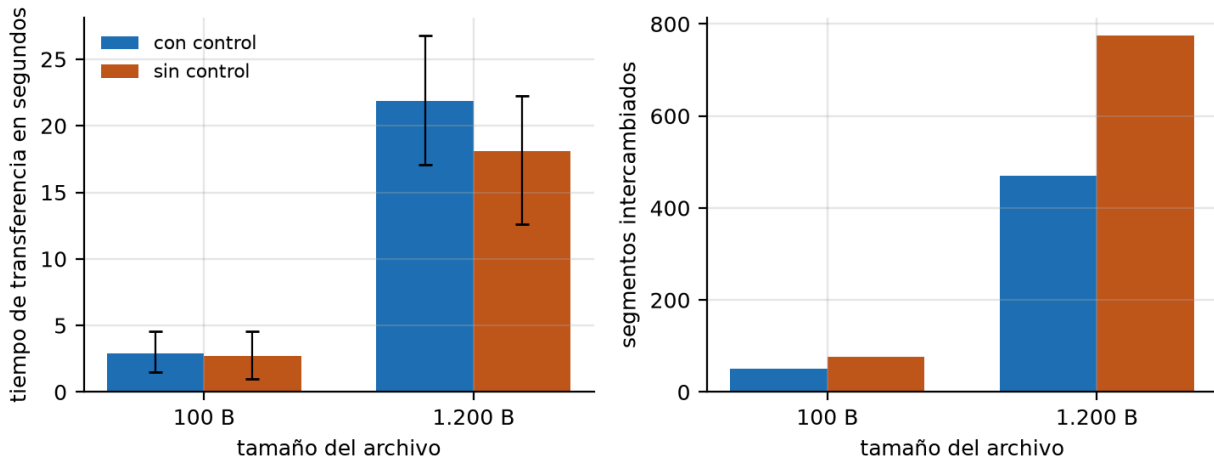


Figura 3: tiempo y segmentos con y sin control de congestión

Sin control de congestión la ventana parte en ocho segmentos, así que el emisor llena el canal desde el comienzo y termina antes. Con control de congestión la ventana parte en uno y vuelve a uno en cada *timeout*, lo que alarga el envío un 6% en el archivo pequeño y un 21% en el grande. A cambio, el tráfico baja de 76,0 a 50,6 segmentos en el caso pequeño y de 774,8 a 469,6 en el grande.

La respuesta a la pregunta del enunciado es que Go-Back-N sin control de congestión resulta más rápido en este escenario, con 18,09 s frente a 21,87 s. Ese resultado se explica porque la prueba corre sobre un enlace local sin competencia, donde la ventana grande no provoca congestión real y solo paga retransmisiones. En una red compartida esa misma ventana castigaría a los demás flujos, que es justamente el problema que el control de congestión viene a evitar.

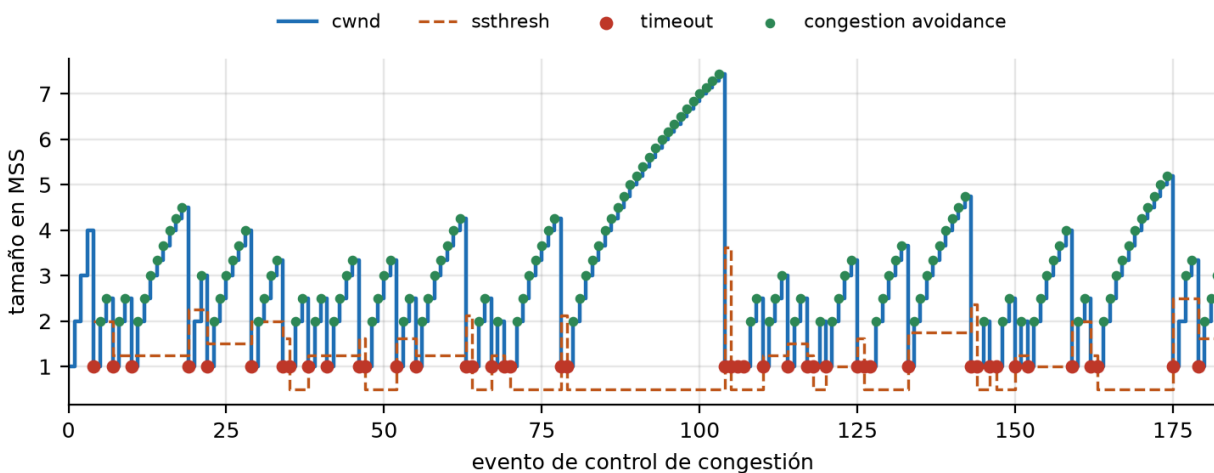


Figura 4: evolución de la ventana de congestión bajo pérdida

La Figura 4 sigue la ventana de congestión durante una transferencia de 1.200 B con un 20% de pérdida. El patrón de sierra aparece con claridad, con el crecimiento exponencial de *slow start*, la caída a un MSS en cada *timeout* y los tramos lineales de *congestion avoidance*. El umbral desciende hasta valores bajos cuando los vencimientos se encadenan, lo que mantiene la ventana pequeña durante buena parte del envío.

Conclusión

Las dos actividades quedaron operativas y el conjunto transfiere archivos sin pérdida de datos sobre un canal que descarta uno de cada cinco segmentos. El trabajo mostró que la parte difícil no está en los algoritmos sino en los estados intermedios, como el handshake a medio terminar o el timer que nadie detiene. El control de congestión cumple su papel al reducir el tráfico cerca de un 40%, aunque su beneficio se vuelve visible solo cuando la red tiene otros flujos que proteger.

Bibliografía

Bachmann, I. (2026a). Actividad: sockets orientados a conexión con Stop & Wait [Material del curso Redes (CC4303)]. Universidad de Chile.

Bachmann, I. (2026b). Actividad: TCP con control de congestión [Material del curso Redes (CC4303)]. Universidad de Chile.

Kurose, J. F., y Ross, K. W. (2013). Computer networking: A top-down approach (6a ed.). Pearson.